[Nam Le] Toolchain & Cross-Compilation

1. What is/ is inside a cross-compiling toolchain?

1.1 What is Host system & Target system ?

In Embedded Linux, we work on 2 different systems

- **Host system**: (Maybe) Our laptop, run **Ubuntu**, compiling and running the programming (Ex: **x86-64**)
- **Target system**: Embedded board is run by Linux, execute the program is built for this architecture and environment. (Ex **ARMv7**)

Note: Host and Target is the purposion in development steps, no force to different between 2 physical system. They maybe same in architecture or not same in architecture

1.2 Native compilation & Cross-Compilation + Toolchain

Native Compilation:
We compile the program directly on system which we want to run on
Example:

```
We have main.c on Ubuntu x86-64
--> gcc main.c -o main
--> main - executable x86-64
```

GCC on Ubuntu x86-64 will create the program for this x86-64 environment

Toolchain is a set of tools that compiles source code into executables that can run on your target device and includes a compiler, a linker, and runtime libraries

Cross Compilation:
We compile the program on this system but create a executable file to execute on another system
Example:

```
arm-linux-gnueabihf-gcc main.c -o main
--> main - executable ARM
```

So **cross-compilation** is the compiler which **architecture** of the system do the compiler **differ from** destination architecture which compiler compile

1.3 Why Embedded Linux need Cross-Compilation?

1. First, the limitation of board. Slow CPU, small RAM and small ROM
2. Second, full environment for development. Host has **IDE, debugger, analization tool and dependencies** for build
3. Third, target maybe not have compiler

--> Use the power of my PC laptop and remain the environment development **isolated from** board device **Note**: If board have enough resource and corresponding compiler, we also can compile directly on board

1.4 The base components of GNU Toolchain

There are 4 main components:

1. Binutils (Binary Utilities): Including the tools handling the binary files - assembler, linker and some utilities (objdump, readelf, strip,...)
2. GNU Compiler Collection (GCC): These are the compilers for C and other languages, compile to assembly code
3. C library: (POSIX API) With the purpose is the communication between application and Kernel Linux. There are 4 main components:

- `libc`: standard C library (contain POSIX function `printf`, `open`, `read`, `write`,...)
- `libm`: Math functions (`cos`, `exp`, `log`,...)
- `libpthread`: Manage the thread (POSIX thread)
- `librt`: For runtime (shared memory, asynchronous I/O)

4. Linux Kernel Header Sanitized: **Linux Kernel header** is sanitized for compile C library and application need to communicate directly with Kernel

1.5 Identification Structure for Toolchain (Prefix)

The prefix of a Toolchain often specified as **Tuple** structure, including 3 or 4 components has type `CPU-Vendor-Kernel-OS` (Example: `arm-linux-gnueabihf` or `x86_64-linux-gnu)

- CPU: Target CPU (ARM, MIPS, x86-64,...)
- Vendor: Toolchain supplier (`buildroot`, `poky`, or `unknown`)
- Kernel: alway Linux
- OS/ABI: User environment and Application binary interface (`gnu`, `musl`, `gnueabi`, `gnueabihf`,...)

Note:
`gnu`: for GNU/Linux OS
`gnuaebi` = `gnu`+ `eabi`: Embedded Application binary interface
`gnuaebihf` = `gnu` + `eabi` + `hf`: Hard Float (chip has `FPU` - Floating Point Unit)

1.6 Choosing a C library

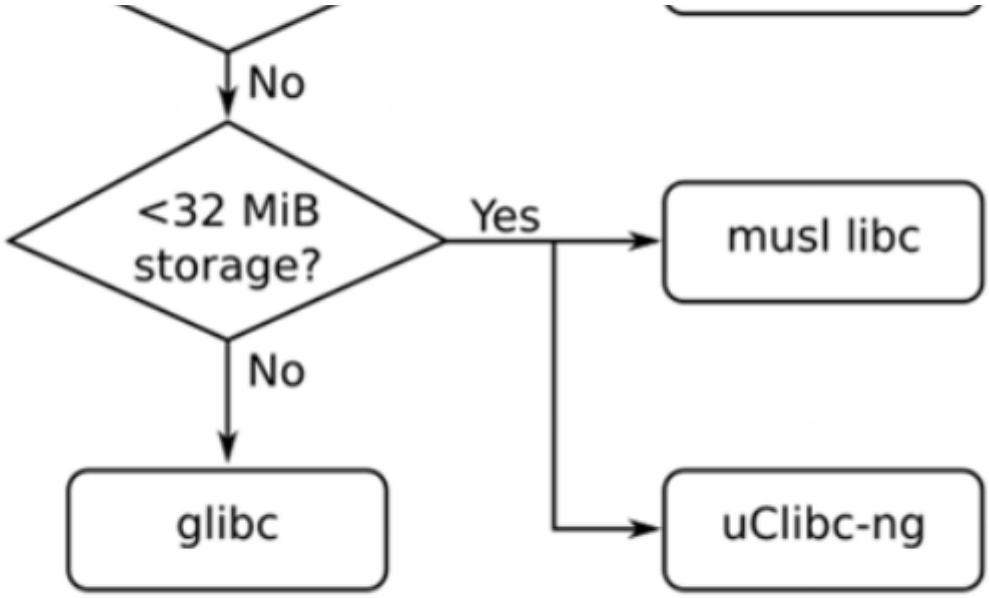


Figure 2.2 – Choosing a C library

To choose the C library:

- **glibc**: Standard C GNU library, adequate (full) but big capacity
- **musl libc**: Small, standards-compliant, good-choice for systems with a limited amount of RAM and storage
- **uClibc-ng**: for microcontroller device or small system (ng-next generation)
- *eglibc*: **Stop develop and gather to** `glibc`

1.7 Definition of Sysroot

Sysroot is the directory (maybe) place inside **toolchain**, containing subdirectories to **serve for compiler and linker**

- `lib`: Contain **shared objects** (`.so`) of C library, and dynamic linker `ld-linux`
- `usr/lib`: Contain **static libraries** and addition libraries
- `usr/include`: Contain file header
- `usr/bin` & `sbin`: Contain the utilities run on target device

2. Ready-to-use cross-compiling toolchains vs. building a custom toolchain

Perform an example with Raspberry PI 4B 64bit

2.1 Ready-to-use cross-compiling toolchains

On linux 22.04 LTS, there are available all Cross-Compilation Toolchain package for arrch64 (Pi OS 64 bit)
We can get it by `sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu`
Example:

```
#include <stdio.h>
int main() {
    printf("Hello World from Pi 4B\n");
    return 0;
}
```

Cross-compile: `aarch64-linux-gnu-gcc hello.c -o hello`
Then, push this binary file to Target system (Pi): `scp hello namlh@10.0.0.14:/home/namlh/cross_compile`
Execute it on Pi : `namlh@raspberrypi:~/cross_compile $ ./hello`
Result: `Hello World from Pi 4B`

2.2 Building a custom toolchain

Follow the instruction of Documentation
The result is same with the Ready-to-use toolchain

3. Why the kernel headers in the toolchain must not be newer than the kernel that will run on the target

The purpose of Kernel Headers in Toolchain
Kernel header containing the definition of data structure, const, and system call to directly access to Kernel. They are necessary for application communicate with device drivers

Why Kernel headers not be newer than kernel on target

- **Backward Compatibility** of Linux Kernel: System call interface of Linux Kernel, always ensure the **backward compatibility**. It means that newer Kernel on Targer still understand and more support when system calls are definitied from older Kernel Header version
- **Risk** when use newer Kernel header: C lib or application will be compiled with new app/system interface. When app is executed on Target, these syscall **have not already installed or supported**, occurs **runtime error**

The rule when choose Kernel Header version

- No force Kernel Header is same 100% with Kernel on Target
- But, it's force to Kernel Header in Toolchain must be **same as, or older than** Kernel version on Target

4. The ELF interpreter (dynamic loader) path is fixed inside the binary

When we build the exe file with a specific architecture, in ELF header section will contain suitable of shared lib dir:
For example, Reading the ELF section of aarch64 exe file by command: `readelf -l`

```
INTERP         0x0000000000000238 0x0000000000400238 0x0000000000400238
                 0x000000000000001b 0x000000000000001b  R      0x1
      [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
  LOAD           0x0000000000000000 0x0000000000400000 0x0000000000400000
                 0x00000000000007cc 0x00000000000007cc  R E    0x10000
  LOAD           0x000000000000fdd8 0x000000000041fdd8 0x000000000041fdd8
                 0x0000000000000258 0x0000000000000260  RW     0x10000
```

As we can see, this exe file request the shared lib file of aarch64 architecture.

If we want to build and run the exe file of another kernel library like from gnu to uClibC, we must use the flag `-static` when build exe file. After that, the exe file will contain all necessary function definition inside without using it in shared lib.

Revision #6
Created 2026-10-20 06:46:26 UTC by Lê Hoài Nam
Updated 2026-10-21 10:29:31 UTC by Lê Hoài Nam